# Notebook de BolitosFC

Camilo Sodo, Gaspar Hilman, Juan Ignacio Varisco

2026

# Índice general

**1. Generalidades** **1**
1.1. Plantilla . . . . . 1
1.2. Makefile . . . . . 1
1.3. Busqueda Binaria . . . . . 2

**2. Grafos** **3**
2.1. Recorridos . . . . . 3
2.2. Diametro en Arboles . . . . . 4
2.3. Dijkstra . . . . . 5
2.4. Ascenso Binario . . . . . 5
2.5. Primer Ancestro Común . . . . . 6
2.6. Componentes Furtemente Conexas . . . . . 7
2.7. Ordenamiento Topologico . . . . . 8

**3. Teoria de Números** **10**
3.1. Minimo Comun Multiplo/Divisor Comun Menor . . . . . 10
3.2. Criba de Eratóstenes . . . . . 10
3.3. Exponenciación Binaria . . . . . 11

# Generalidades

## 1.1. Plantilla

Plantilla de código para competencias.

```cpp
#include <bits/stdc++.h>
#define ll long long
#define dd long double
#define ull unsigned long long
#define forr(i, h) for (ll i = 0; i < h; i++)
#define forrr(i, d, h) for (ll i = d; i < h; i++)
#define techo(x, k) ((x + k - 1) / k)
#define endl '\n'
#define initArr(arr, largo, contenido)

    \
    for (int i = 0; i < largo; i++) arr[i] = contenido;
using namespace std;
using Par = pair<ll, ll>;
using GrafoPesado = vector<vector<pair<ll, ll>>>;
using Grafo = vector<vector<ll>>;
using Digrafo = vector<vector<pair<ll, bool>>>;
using Arbol = vector<vector<ll>>;

int main() {
#ifdef LOCAL
    freopen(FILE_DIR, "r", stdin);
#endif
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    ll casos;
    while (cin >> casos) {
        while (casos--) {
        }
    }
}
```

## 1.2. Makefile

Makefile para compilar el codigo mas comodamente.

```
SHELL := /bin/bash
FILE_NAME := $(basename $(notdir $(FILE)))
DIR_NAME := $(dir $(FILE))
OUT_NAME := $(DIR_NAME)$(FILE_NAME).out
IN_NAME := $(DIR_NAME)$(FILE_NAME).in
```

```
6
7  run: compile
8    ./$(OUT_NAME)
9
10 debug: compile
11   gdb -x ./comandosGDB ./$(OUT_NAME)
12
13 compile: $(FILE)
14   g++ $(FILE) -DLOCAL -DFILE_DIR='"./$(IN_NAME)"' -g -fsanitize=address,
       undefined -o $(OUT_NAME) -std=c++17
15
16 clear:
17   rm *.out
18
19 fill:
20   mkdir $(DIR)
21   cat template.cpp | tee $(DIR)/{a..n}.cpp > /dev/null
22   touch $(DIR)/{a..n}.in
23
24 traer:
25   cp $(DIR)/.clang-format .clang-format
26   cp $(DIR)/makefile makefile
27   cp $(DIR)/comandosGDB comandosGDB
28   cp $(DIR)/template.cpp template.cpp
29
30 llevar:
31   cp .clang-format $(DIR)/.clang-format
32   cp makefile $(DIR)/makefile
33   cp comandosGDB $(DIR)/comandosGDB
34   cp template.cpp $(DIR)/template.cpp
```

## 1.3. Busqueda Binaria

Plantilla de una busqueda binaria basica.

```
1  #define ll long long
2  using namespace std;
3
4  /* Binary search de Gaspi */
5  bool func(ll x);
6
7  ll solve(ll min_val, ll max_val) {
8      ll L = min_val;
9      ll R = max_val;
10     ll ans = -1;
11     while (L <= R) {
12         ll mid = L + (R - L) / 2;
13         if (func(mid)) {
14             ans = mid;
15             R = mid - 1;
16         } else {
17             L = mid + 1;
18         }
19     }
20     return ans;
21 }
```

# Grafos

## 2.1. Recorridos

Plantillas para recorridos BFS y DFS.

```cpp
#include <bits/stdc++.h>
#define ll long long
using namespace std;
using GrafoPesado = vector<vector<pair<ll, ll>>>;
using Grafo = vector<vector<ll>>;
using Digrafo = vector<vector<pair<ll, bool>>>;
using Arbol = vector<vector<ll>>;

void bfs(Grafo &g, ll raiz) {
    vector<bool> visitados(g.size(), false);
    queue<ll> q;
    q.push(raiz);
    while (!q.empty()) {
        ll actual = q.front();
        q.pop();
        visitados[actual] = true;
        for (ll vecino : g[actual]) {
            if (visitados[vecino]) continue;
            q.push(vecino);
        }
    }
}

void dfs(Grafo &g, ll raiz) {
    vector<bool> visitados(g.size(), false);
    stack<ll> p;
    p.push(raiz);
    while (!p.empty()) {
        ll actual = p.top();
        p.pop();
        visitados[actual] = true;
        for (ll vecino : g[actual]) {
            if (visitados[vecino]) continue;
            p.push(vecino);
        }
    }
}

void dfs_recursivo(Grafo &g, ll raiz, vector<bool> &visitado) {
    visitado[raiz] = true;
    for (ll vecino : g[raiz]) {
        if (visitado[vecino]) continue;
        dfs_recursivo(g, vecino, visitado);
```

```
    }
}
```

## 2.2. Diametro en Arboles

Calcula el camino mas largo de un árbol (su diametro).

```
#include <bits/stdc++.h>
#define ll long long
#define forr(i, h) for (ll i = 0; i < h; i++)
using namespace std;

/* Maxima distancia desde un vertice */

using Grafo = vector<vector<ll>>;
vector<ll> caminoDistancia;
ll maximaDistanciaDesde(Grafo &g, ll desde) {
    caminoDistancia = vector<ll>();
    vector<bool> encolado(g.size(), false);
    vector<ll> padres(g.size(), -1);
    queue<ll> q;
    ll lejano;
    q.push(desde);
    while (!q.empty()) {
        ll actual = q.front();
        q.pop();
        lejano = actual;
        for (ll vecino : g[actual]) {
            if (encolado[vecino]) continue;
            q.push(vecino);
            encolado[actual] = true;
        }
    }

    ll actual = lejano;
    while (actual != -1) {
        caminoDistancia.push_back(actual);
        actual = padres[actual];
    }
    reverse(caminoDistancia.begin(), caminoDistancia.end());
    return caminoDistancia.size();
}

/* Calculo de Diametro */

using Arbol = vector<vector<ll>>;
vector<ll> calcularDiametro(Arbol &g, ll nodo) {
    maximaDistanciaDesde(g, nodo);
    ll extremo = caminoDistancia.back();
    maximaDistanciaDesde(g, extremo);
    return caminoDistancia;
}

/* Centro del arbol (medio del diametro) (minimiza distancia maxima)*/

ll calcularCentro(vector<ll> diametro) { return diametro[diametro.size()
    / 2]; }
```

## 2.3. Dijkstra

Busca para todos los vertices la distancia minima desde un punto de partida dado.

```cpp
#include <bits/stdc++.h>
#define ll long long
using namespace std;

vector<ll> distanciasDijkstra;
using GrafoPesado = vector<vector<pair<ll, ll>>>;
using ElementoDijkstra = pair<ll, ll>;
// TODO: testear
void Dijkstra(GrafoPesado g, ll inicial) {
    distanciasDijkstra = vector<ll>(g.size(), 10e10);
    distanciasDijkstra[inicial] = 0;
    priority_queue<ElementoDijkstra, vector<ElementoDijkstra>, greater<
        ElementoDijkstra>> queue;
    queue.push({0, inicial});
    while (!queue.empty()) {
        auto [distanciaActual, actual] = queue.top();
        queue.pop();
        if (distanciasDijkstra[actual] < distanciaActual) continue;
        for (auto [vecino, largoArista] : g[actual]) {
            if (distanciasDijkstra[vecino] > largoArista +
                distanciaActual) {
                distanciasDijkstra[vecino] = largoArista +
                    distanciaActual;
                queue.push({distanciasDijkstra[vecino], vecino});
            }
        }
    }
}
```

## 2.4. Ascenso Binario

En arboles, permite encontrar un ancestro arbitrario de un vertice cualquiera rapidamente.

```cpp
#include <bits/stdc++.h>
#define ll long long
#define forr(i, h) for (ll i = 0; i < h; i++)
using namespace std;

using Arbol = vector<vector<ll>>;
vector<vector<ll>> ancestros; // La raiz es su propio ancestro
vector<ll> profundidad;
void calculaPrimerAncestro(Arbol &g, ll raiz) {
    stack<ll> p;
    p.push(raiz);
    ancestros[raiz].push_back(raiz);
    profundidad[raiz] = 0;
    while (!p.empty()) {
        auto act = p.top();
```

```
        p.pop();
        for (ll vecino : g[act]) {
            if (profundidad[vecino] == -1) {
                profundidad[vecino] = profundidad[act] + 1;
                ancestros[vecino].push_back(act);
                p.push(vecino);
            }
        }
    }
}

void calculaKesimoAncestro(Arbol &g, ll k) {
    forr(i, g.size()) {
        ll ancestro = ancestros[ancestros[i][k - 1]][k - 1]; // Cuidado
            si se cambia el ancestro de la raiz estandar
        ancestros[i].push_back(ancestro);
    }
}

void calcularBinaryLifting(Arbol &g, ll raiz) {
    ancestros = vector<vector<ll>>(g.size());
    profundidad = vector<ll>(g.size(), -1);
    calculaPrimerAncestro(g, raiz);
    for (ll cont = 2, k = 1; cont < g.size(); cont *= 2, k++) {
        calculaKesimoAncestro(g, k);
    }
}

ll enecimoAncestro(Arbol &g, ll nodo, unsigned ll n) {
    for (unsigned ll salto = 0; n != 0; n >>= 1, salto++) {
        if ((n & 1) != 0) {
            nodo = ancestros[nodo][salto];
        }
    }
    return nodo;
}
```

## 2.5. Primer Ancestro Común

En arboles, permite encontrar rapidamente el menor ancestro común (lowest common ancestor) entre dos vertices arbitrarios. Utiliza ascenso binario.

```
#include <bits/stdc++.h>
#define ll long long
#define forr(i, h) for (ll i = 0; i < h; i++)
using namespace std;

using Arbol = vector<vector<ll>>;
vector<vector<ll>> ancestrosLCA;
vector<ll> profundidadLCA;

// Utiliza Binary Lifting. Ver binaryLifting.cpp
void calcularBinaryLifting(Arbol &g, ll raiz); //
ll enecimoAncestro(Arbol &g, ll nodo, unsigned ll n);

void calcularLCA(Arbol &g, ll raiz) { calcularBinaryLifting(g, raiz); }
```

```
15
16 ll lowestCommonAncestor(Arbol &g, ll a, ll b) {
17     if (profundidadLCA[a] < profundidadLCA[b]) swap(a, b);
18
19     a = enecimoAncestro(g, a, profundidadLCA[a] - profundidadLCA[b]);
20
21     while (a != b) {
22         ll L = 0;
23         ll R = ancestrosLCA[b].size() - 1;
24         ll ans = 0;
25         while (L <= R) {
26             ll mid = L + (R - L) / 2;
27             if (ancestrosLCA[a][mid] != ancestrosLCA[b][mid]) {
28                 ans = mid;
29                 L = mid + 1;
30             } else {
31                 R = mid - 1;
32             }
33         }
34         a = ancestrosLCA[a][ans];
35         b = ancestrosLCA[b][ans];
36     }
37     return a;
38 }
```

## 2.6. Componentes Furtemente Conexas

Permite encontrar las componentes fuertemente conexas de un digrafo.

```
#include <bits/stdc++.h>
#define ll long long
#define forr(i, h) for (ll i = 0; i < h; i++)
#define endl '\n'
using namespace std;
using Digrafo = vector<vector<pair<ll, bool>>>;

vector<ll> componente;
vector<bool> visitado;
stack<ll> semiOrden;
#define CENTINELA -1
void visitarKosaraju(Digrafo &g, ll nodo) {
    if (visitado[nodo]) return;
    visitado[nodo] = true;
    for (auto [vecino, salida] : g[nodo]) {
        if (!salida) continue;
        visitarKosaraju(g, vecino);
    }
    semiOrden.push(nodo);
}

void asignarKosaraju(Digrafo &g, ll nodo, ll comp) {
    if (componente[nodo] != CENTINELA) return;
    componente[nodo] = comp;
    for (auto [vecino, salida] : g[nodo]) {
        if (salida) continue;
        asignarKosaraju(g, vecino, comp);
```

```
    }
}

// Determina componentes fuertemente conexas de un digrafo (aristas [vecino,
// salida?])
void algoritmoKosaraju(Digrafo &g) {
    componente = vector<ll>(g.size(), CENTINELA);
    visitado = vector<bool>(g.size(), false);
    forr(nodo, g.size()) { visitarKosaraju(g, nodo); }
    ll nroComponentes = 0;
    while (!semiOrden.empty()) {
        ll nodo = semiOrden.top();
        asignarKosaraju(g, nodo, nroComponentes);
        nroComponentes = max(nroComponentes, componente[nodo] + 1);
        semiOrden.pop();
    }
}
```

## 2.7. Ordenamiento Topologico

Ordena los vertices de un DAG (digrafo aciclico dirigido) de forma que toda arista e = (v->w) cumple que v precede a w en el orden. Sirve para determinar si el grafo no es aciclico.

```
#include <bits/stdc++.h>
#define ll long long
#define forr(i, h) for (ll i = 0; i < h; i++)
#define endl '\n'
using namespace std;

using Digrafo = vector<vector<pair<ll, bool>>>;
vector<ll> ordenTarjan;
vector<bool> visitadoTarjan;
vector<ll> marcasTarjan;
ll ordenadosTarjan;
bool visitarTarjan(Digrafo &g, ll nodo, ll marcaActual) {
    if (visitadoTarjan[nodo]) return true;
    if (marcasTarjan[nodo] == marcaActual) return false;
    marcasTarjan[nodo] = marcaActual;
    bool hayCiclo = false;
    for (auto [vecino, salida] : g[nodo]) {
        if (!salida) continue;
        hayCiclo = hayCiclo && !visitarTarjan(g, vecino, marcaActual);
    }
    visitadoTarjan[nodo] = true;
    ordenTarjan[nodo] = ordenadosTarjan++;
    return !hayCiclo;
}

bool ordenamientoTarjan(Digrafo &g) {
    ordenTarjan = vector<ll>(g.size());
    visitadoTarjan = vector<bool>(g.size(), false);
    marcasTarjan = vector<ll>(g.size(), -1);
    ordenadosTarjan = 0;
    bool esAciclico = true;
    for (ll nodo = 0; nodo < g.size() && esAciclico; nodo++) {
```

```
33          if (visitadoTarjan[nodo]) continue;
34          esAciclico = visitarTarjan(g, nodo, nodo);
35      }
36      return esAciclico;
37  }
```

# Teoria de Números

## 3.1. Minimo Comun Multiplo/Divisor Comun Menor

Calcula el mcm o dcm de dos numeros.

```
#include <bits/stdc++.h>
#define ull unsigned long long
using namespace std;

ull dcmEuclideano(ull n1, ull n2) {
    if (n1 == n2 || n1 == 0 || n2 == 0) return max(n1, n2);
    if (n2 < n1) swap(n1, n2);
    return dcmEuclideano(n1, (n2 % n1));
}

ull mcmEuclideano(ull n1, ull n2) { return (n1 / dcmEuclideano(n1, n2))
   * n2; }
```

## 3.2. Criba de Eratóstenes

Calcula para todos los numeros menores a un N si son primos y su menor divisor.

```
#include <bits/stdc++.h>
#define ll long long
#define forr(i, h) for (ll i = 0; i < h; i++)
using namespace std;

bool es_primo(int x) {
    for (int i = 2; i < x / 2; i++) {
        if (x % i == 0) {
            return false;
        }
    }
    return true;
}

ll criba(ll MAX, vector<bool> &primos, vector<ll> &minDivisor) {
    primos.clear();
    minDivisor.clear();
    forr(i, MAX + 1) {
        primos.push_back(true);
        minDivisor.push_back(i);
    }

    primos[0] = primos[1] = false;

```

```
    ll cantPrimos = MAX;
    for (int p = 2; p * p <= MAX; p++) {
        if (!primos[p]) continue;
        for (int d = p * p; d <= MAX; d += p) {
            if (primos[d]) {
                primos[d] = false;
                cantPrimos--;
                minDivisor[d] = p;
            }
        }
    }
    return cantPrimos;
}
```

## 3.3. Exponenciación Binaria

Calcula rapidamente la enecima potencia de un numero. Util para conseguir el inverso modular.

```
#include <bits/stdc++.h>
#define ll long long
using namespace std;
#define MOD ((ll)1e6 + 7)
#define sum(a, b) (((a % MOD) + (b % MOD)) % MOD)
#define res(a, b) (((a % MOD) - (b % MOD) + MOD) % MOD)
#define mul(a, b) (((a % MOD) * (b % MOD)) % MOD)

ll exponenciacionBinaria(ll x, ll e) {
    ll acumulador = 1;
    ll potBinaria = x;
    while (e != 0) {
        if (e & 1) {
            acumulador = mul(acumulador, potBinaria);
        }
        e = e >> 1;
        potBinaria = mul(potBinaria, potBinaria);
    }
    return acumulador;
}

ll inversoModular(ll x) { return exponenciacionBinaria(x, MOD - 2); } // MOD DEBE ser primo
```